

preply.

ANALYTICS ENGINEERING · TAKE-HOME

# Analytics Engineer Case Study

A real problem from the PayOps team — modeling breakage at payment level with a daily estimation challenge.

## WELCOME

Thanks for taking the time to work through this case. It's designed to reflect the kind of problem you'd actually pick up in your first few months on the team — ambiguous enough that you'll need to make judgment calls, concrete enough that you can ship something.

We expect this to take **around 4-6 hours of focused work**. We're not looking for polish or completeness — we're looking for clear thinking, sound modeling choices, and honest trade-offs. If you run out of time, write down what you'd do next instead of trying to cram it in.

You're welcome (and encouraged) to use AI assistants. One of the deliverables asks you to reflect on how you used them.

## Business context

Preply is a subscription-based online tutoring marketplace. Students book lessons with tutors; the platform handles payments, scheduling, and payouts.

### How subscriptions work

- Students buy a plan for a fixed number of tutoring hours (4 to 20 hours).
- A student's **subscription cycle is 28 days long, anchored on the date of their first payment**. For example, a student whose first payment is on January 5th will have cycles that reset on February 2nd, March 2nd, March 30th, April 27th and so on. Different students have different cycle anchor dates.
- At the end of each cycle, the plan auto-renews and a new cycle begins with a fresh allocation of hours.
- Hours are "burned" when a student **books** a lesson — the moment a booking is made, the corresponding hours are deducted from the student's balance. For this case, assume bookings are final: no cancellations, no rescheduling, no no-shows.
- Any hours still unbooked at the end of the 28-day cycle **expire**. They don't roll over.

### How Preply makes money

Preply earns revenue from two sources:

1. **Commission** — 20% of the value of hours that students actually book (the remaining 80% goes to the tutor).
2. **Breakage** — 100% of the value of hours that remain unbooked at the end of a cycle.

For this case, you can ignore discounts and promotional credits.

## THE CHALLENGE WITH BREAKAGE

Breakage revenue is only known *once the 28-day cycle ends*. Until then, we don't know how many more hours the student will book. But the Payment Operations team (PayOps) needs to monitor revenue in real time — they can't wait 28 days to see what happened.

### Example:

```
A student makes their first payment on January 5th:
$80 for an 8-hour plan at $10/hour
Cycle ends on February 1st (28 days later)

Between January 5th and February 1st, they book 6 hours of lessons.
2 hours remain unbooked when the cycle closes.

Commission = 20% × 6 booked hours × $10 = $12
Breakage    = 2 unbooked hours × $10      = $20
Total revenue                                = $32 (recognized Feb 1st)

A new cycle begins on February 2nd with 8 fresh hours, ending March 1st.
```

## Your task

The PayOps team has asked you to build a **daily updated breakage estimation** per payment, that gives them a best-guess view of total expected breakage revenue, refreshed every day, for all payments that are still inside their open 28-day cycle.

Concretely, for every payment in the last 28 days, we need a daily estimate of how much breakage it will ultimately produce. Once the cycle closes, the estimate is replaced by the actual number. The PayOps team wants to open their dashboard every morning and see, for any payment (or any day, student segment, plan size, etc.), both:

- **Actual breakage** for payments whose cycles have closed
- **Estimated breakage** for payments still inside their 28-day window, updated daily as the cycle progresses

Your job is to design and build the model and dashboard that make this possible.

## What we'd like you to deliver

### 1. A data model

Build/suggest the dbt models (or SQL transformation pipeline with equivalent structure) that produces the mart powering the dashboard. You decide the grain, the layering, and the modeling approach.

We've provided a synthetic dataset:

| Table                     | Description                                                                          |
|---------------------------|--------------------------------------------------------------------------------------|
| <code>raw_payments</code> | One row per subscription payment — payment date, student, plan size, price per hour  |
| <code>raw_lessons</code>  | One row per lesson booked — booking date, student, hours booked                      |
| <code>raw_students</code> | Student metadata — country, signup date, acquisition channel, persona, first subject |

The dataset covers ~12 months of activity. You don't need to use every column or every table.

### At minimum, we expect:

- Clean staging models over the raw data

- Intermediate models that encode the business logic (when a cycle starts and ends for each student, how bookings map to cycles, what counts as a closed cycle, etc.)
- A final mart suitable for the dashboard
- Basic tests (uniqueness, not-null, and any business-logic tests you think matter)

#### A NOTE ON TIMING

This case uses a fixed point-in-time view of the data. **Assume today is the last day covered by the dataset.**

Payments made in the 28 days prior are inside their open cycle and are the ones requiring estimation — their actual breakage isn't known yet. Cycles from earlier payments have already closed and their breakage is known.

## 2. A dashboard mockup

Build a dashboard mockup the PayOps team could actually use day-to-day. Use any tool you're comfortable with — Figma, Looker, Tableau, Streamlit app, even a polished .html if you'd rather focus your time on the modeling.

The dashboard should answer questions like:

- *What's our total expected breakage revenue this month? How does it split between actual and estimated?*
- *How has the estimate for a given payment cohort evolved over time as more information became available?*
- *Are there segments (plan size, country, new vs. renewing students) where breakage is systematically higher or lower?*

Don't feel obliged to build exactly these — build what you think the PayOps team would find most useful. Use your judgment.

## 3. A summary presentation

A slides presentation or a README covering:

- **Your approach** — how you decided to estimate breakage and why
- **The data model** — a diagram or brief description of the layers and grain
- **Key assumptions and trade-offs** — things you had to decide without enough information
- **Caveats and limitations** — what you'd warn the PayOps team about before they use this
- **What you'd do next** — if you had another week on this, what would you build or improve?

## 4. An AI usage note

One paragraph or a few bullets describing:

- Where AI assistants helped you
- Where they gave you something you rejected or corrected
- How you verified the output

#### ON AI

We expect you to use AI. We want to see how you use it well.

## The follow-up conversation

After you submit, we'll schedule a **60-minute session** where you'll walk us through your work and we'll discuss it together. Come prepared to talk about:

- Why you chose the estimation approach you did, and what alternatives you considered
- The grain of your final mart and the reasoning behind it
- How your model would behave on the edge cases (a payment on day 1 of its cycle, a payment on day 27, a payment where the student has already booked all their hours)
- How you'd extend this if the PayOps team came back and asked for breakage estimated at the *daily* level

(e.g., "how much breakage do we expect to recognize tomorrow?") rather than the payment level

- What would break or degrade if Preply changed the subscription cycle from 28 days to 30 days
- Anything in the prompt you'd push back on if this were a real stakeholder request

## What we're evaluating

| Dimension         | What we're looking for                                                                   |
|-------------------|------------------------------------------------------------------------------------------|
| Problem framing   | How you interpret the prompt and what questions you ask yourself before modeling         |
| Data modeling     | Grain, layering, clarity, and whether the structure would hold up as requirements evolve |
| SQL & dbt craft   | Readable queries, sensible materializations, useful tests                                |
| Business sense    | Connecting the technical work to the actual decisions PayOps needs to make               |
| Dashboard design  | Clarity, information density, and fit to the audience                                    |
| Communication     | How well your README lets a teammate pick up where you left off                          |
| Critical thinking | Trade-offs made explicit, caveats stated honestly, follow-ups identified                 |
| AI usage          | Using AI as a thinking partner rather than an oracle                                     |

## Submission

Please send us:

- Your dbt models (or equivalent SQL files), via a link to a Git repo, or the files themselves
- The dashboard link, screenshots, or mockup file
- Your Slides/README including the AI usage note

### QUESTIONS?

If anything in the prompt is unclear, just ask — it's not a trick. We'd rather answer a question now than have you spend two hours on the wrong interpretation.

Good luck, and have fun with it!